

COMS W4156 Advanced Software Engineering (ASE)

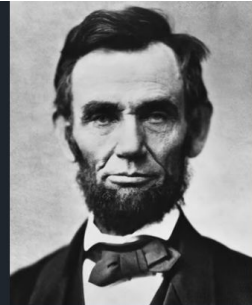
October 3, 2024

On this day ...



Thanksgiving Day Designated

1863 US President [Abraham Lincoln](#) designates last Thursday
in November as Thanksgiving Day



16th US President
[Abraham Lincoln](#)

Does Every Team Know Who Their Mentor Is?

See <https://static.us.edusercontent.com/files/cyH1Otdpf1wAZy6KfGjdmz0c>

Please arrange to meet with your Mentor asap, see courseworks home page for office hours information



Where Is The Courseworks Home Page?

[This](#) is what we mean by the courseworks home page

It should be the first page you see when you connect to this course in courseworks. Some of your other courses might have an analogous homepage, some might not, since courseworks allows multiple choices for homepage

Choose Course Home Page ×

Select what you'd like to display on the home page.

- ☐ Course Activity Stream
- ☒ Pages Front Page *COMS W4156 Advanced Software Engineering ("ASE")* [\[Change\]](#)
- ☐ Course Modules
- ☐ Assignments List
- ☐ Syllabus

Cancel Save

Agenda

1. **Working in Teams**
2. project management tools demo: Shivansh Srivastava
3. if time permits, Pair Programming
4. Coming Up



Software Engineers Always Work In Teams

As discussed at the beginning of the semester (September 5), software engineering teams always have at least three members: you, the past you, and the future you

This "team" motivates...

commit log - When did I write this code and why?

coding style - I can't read this code

code documentation - What does this code do?

project management - What should I work on next?

and other team methodologies and technologies

....even if your team never grows



Multi-Member Teams

Software engineering teams usually have more members, which mandate...

commit log - Someone changed my code, who did it, when and why?

coding style - I can't read this code

code documentation - What does this code do?

project management - Who is working on the code that my code depends on and when should I expect it to be ready?

and other team methodologies and technologies

...particularly as team grows or churns



Commit Messages (on Main Trunk)



Project History: Commit messages serve as a historical record for code changes and facilitate better understanding and tracking of the project's evolution. Ideally include links to relevant "issues", "tickets", etc. in project management system

Code Review: Help reviewers to quickly grasp the intent behind pull requests, making the review process more efficient and thorough (see 9/26 lecture)

Collaboration: Inform other team members about modifications made by others, enhancing coordination and reducing conflicts

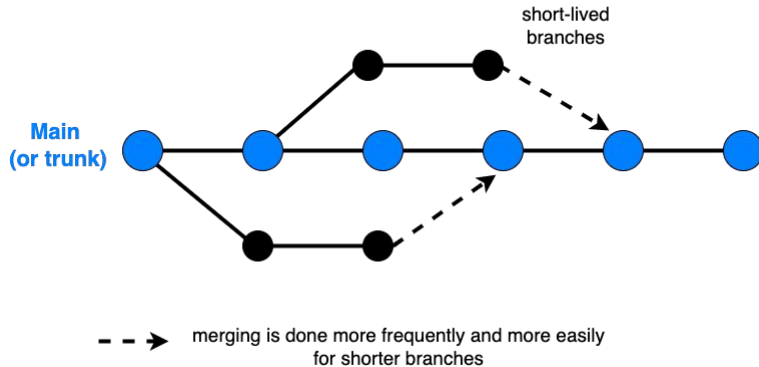
Documentation: Serve as a quick reference, complementing official documentation and helping onboard new team members

First line should be a summary, with explanation and important details following. See examples [here](#)

Commit Logs for Your Team Projects

Trunk-based development

StatusNeo



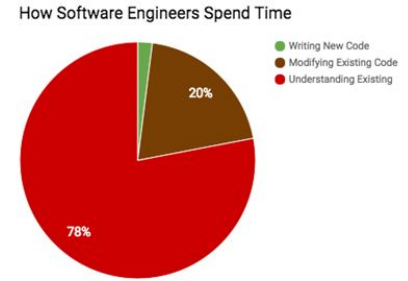
All commit messages to the main branch, except for the initial repository setup commit(s), should say something meaningful about what was added/removed/changed

Label as feature addition, bug fix, refactoring, documentation update, etc.

Cross-reference to your team's choice of project management system (JIRA, Trello, etc.). Use the free tier! 30-day free trials will end too soon

Coding Style

Software engineers spend much more time reading and editing existing code than writing new code. Compliance with a team's coding style helps other developers (and your future self) understand your code



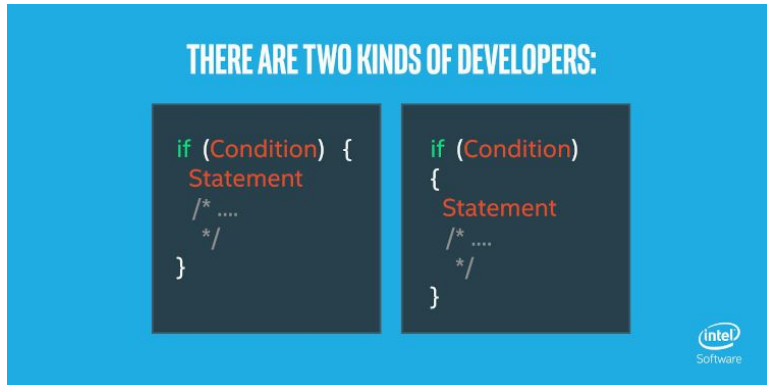
Governs how and when to use comments and whitespace (indentation, blank lines), proper naming of variables and functions (e.g., camelCase, snake_case), code grouping and organization, file/folder structure. See "Clean Code" demo from 9/12

The merge features of git and other version control systems depend on style compliance to avoid superficial, format-related merge conflicts - so developers should always run style checking and fix any problems *before* submitting pull requests

Coding Style for Your Team Projects

Each team should choose some reasonable coding style to be used by everyone on the team

Make sure it has a usable and actively maintained style checking tool that detects and reports any deviations from your chosen style



Code Documentation at Unit Level

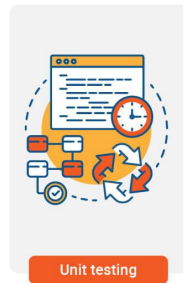
Non-trivial functions should be self-documenting with mnemonic identifiers and, when non-obvious, *inline* comments that explain “why” the code is written this way and/or “why” it works this way - intended audience is developers who need to understand or modify this code

Non-trivial functions exported outside a modular unit (class) should also have a header [javadoc](#), [doxygen](#), etc. comment that explains “what” the function does and “how” to use it (and how not to use it) - intended audience is developers whose code needs to use this code



Unit Tests as Documentation

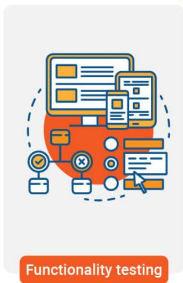
Every significant function behavior should have several test cases to verify that behavior



Unit testing



Integration testing



Functionality testing

These test cases themselves are documented with comments that explain:

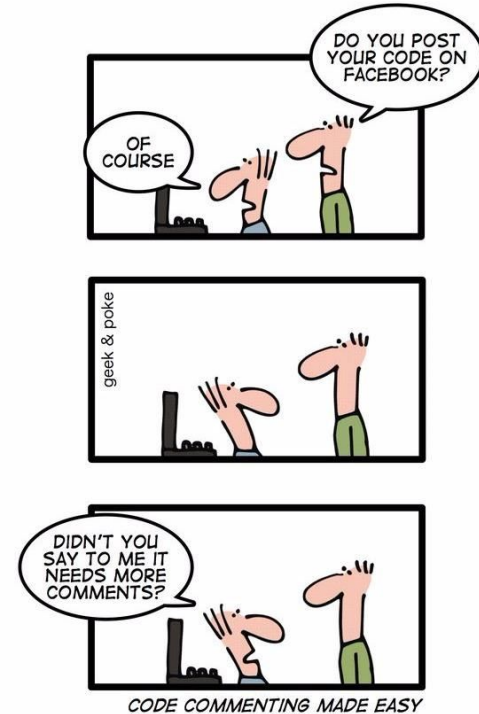
- what environment or context needs to be set up before calling the function (and possibly torn down after it returns),
- what arguments the test provides to the function and what they represent (e.g., equivalence class or boundary, discussed next week),
- what values the test expects the function to return (if any),
- what non-local and/or external data the function reads and writes, and how it is mocked for the test case,
- what exceptions the function can throw or errors it can return,
- and any “gotchas” or restrictions on when and where to use this function

More on choosing test inputs and unit testing next week

Code Documentation for Your Team Projects

Your team mentor will spot-check the code and test documentation in your main branch. Make sure it exists, and is readable, meaningful and accurate

Your team repository must also contain a top-level [README.md](#) that documents your API (or links to external documentation of your API). Some material on API documentation in 9/24 lecture, more coming soon



Agenda

1. Working in Teams
2. project management tools demo: Shivansh Srivastava
3. if time permits, Pair Programming
4. Coming Up



Shivansh Srivastava Project Management Tools Demo

[demo slides](#)



Agenda

1. Working in Teams
2. project management tools demo: Shivansh Srivastava
3. if time permits, Pair Programming
4. Coming Up



Multi-Member Engineering Teams Enable Pair Programming!

Ought to be called “pair software engineering”, but “pair programming” sounds better

Used for many software engineering activities, not just programming

Many companies use pair-programming interviews between job candidate and evaluator (current employee) even if their employees do not pair-program routinely



Ideal Pair Programming

Two people sit side by side at same computer

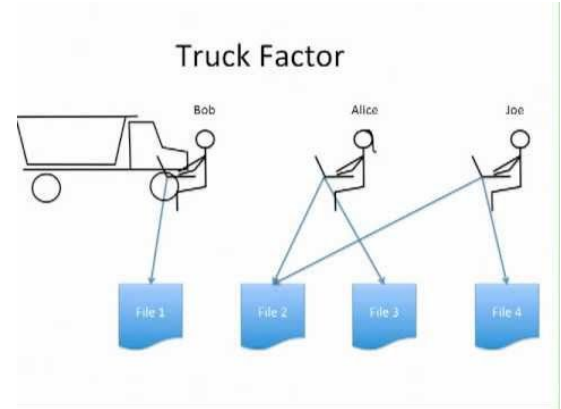
- Take turns “driving” (typing) vs. “navigating” (continuous review of code or whatever artifacts the pair is working on)
- Switch roles at frequent intervals (e.g., [25 minutes](#))
- The pair does **not** divide up the work, they try to do everything **together**
- If necessary to work separately on something, e.g., when one partner is unavailable, they review together
- Nowadays, might use remote desktop



Benefits of Pair Programming

Reduce risk through collective code ownership and maximize “truck factor” = number of engineers that would have to disappear before project would be in serious jeopardy

- (Usually) More productive
- Shared responsibility to complete tasks on time
- Stay focused and on task
- Less likely to read email, surf web, etc.
- Less likely to be interrupted
- Partners expect “best practices” from each other
- Two people can solve problems that one couldn’t do alone and/or produce better solutions
- Pool knowledge resources, improve skills



Pair Programming Code Review

Instant code review happens on the fly during pair programming, where the navigator reviews while the driver writes/edits



Pair programming styles



Unstructured

In unstructured pair programming, the developers can trade off who takes the lead, and should discuss decisions about the code.



Driver/Navigator

In the driver/navigator approach to pair programming, one developer sets the architectural or strategic direction, and the other implements these decisions as code.



Ping-pong

Ping-pong pair programming shifts rapidly back-and-forth between the two developers, like a game of ping pong, where the software is the ball.



Pair Programming for Your Team Projects

We strongly recommend pair programming for your team projects. This is why four-member teams are preferred, since triplet programming does not work very well

But teams with odd numbers of members can switch off pairing



Agenda

1. Working in Teams
2. project management tools demo: Shivansh Srivastava
3. if time permits, Pair Programming
4. **Coming Up**



Upcoming Deadlines

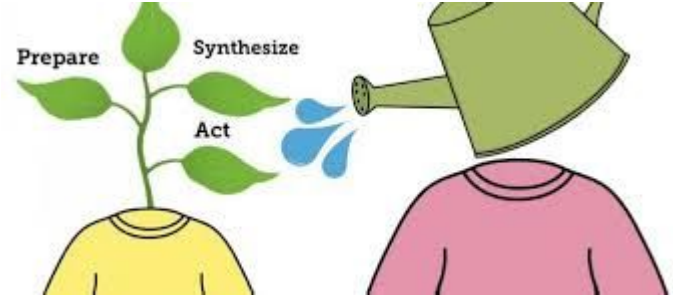
Meet with your Mentor as soon possible!

[Team First Iteration](#) due Friday October 18

[Team First Iteration Demo](#) due Friday October 25

First Assessment Friday October 25th 12:01am
through Monday October 28th 11:59pm -

We will discuss what you should expect when closer,
just one hint now: don't neglect the Reading assignments!



Next Week

finish anything left over from today

Choosing Test Inputs

Unit Testing and Mocking, with demo





Ask Me Anything